

# SVM – Support Vector Machines (Maszyny wektorów wspierających)

**Aneta Ptak-Chmielewska**

Instytut Statystyki i Demografii

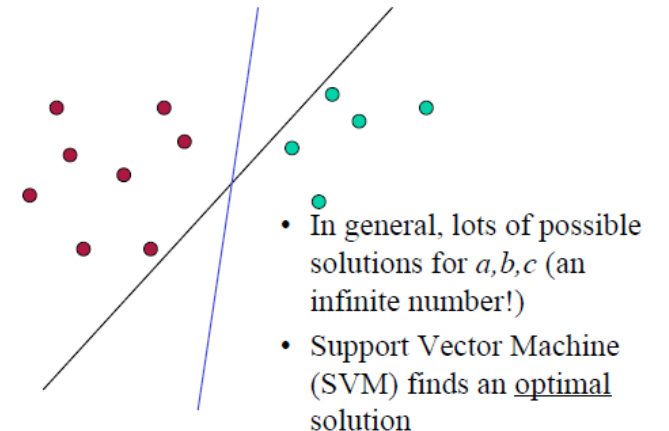
[aptak@sgh.waw.pl](mailto:aptak@sgh.waw.pl)

# Geneza metody SVM

- Vapnik wprowadził nowe rozwiązanie problemu klasyfikacji, za pomocą hiperpłaszczyzn dokonując uogólnienia idei klasyfikacji.
- Metoda Vapnika polega na rozwiązaniu zagadnienia optymalizacji z kwadratową funkcją celu oraz ograniczeniami liniowymi.
- SVM posiada wiele cennych własności takich jak: brak wymagań dotyczących rozkładu zmiennych objaśniających, nieliniowość modelu, niski poziom błędu predykcji na zbiorze testowym, odporność na ewentualne szumy w zbiorze treningowym.
- Działanie maszyny jest w dużym stopniu zautomatyzowane i w konsekwencji wyniki nie są całkowicie interpretowalne.
- Zastosowanie SVM w celu dokonania predykcji wykazuje się dużą dokładnością, pomimo, że nie pokazuje całkowitych relacji pomiędzy zmiennymi w modelu.

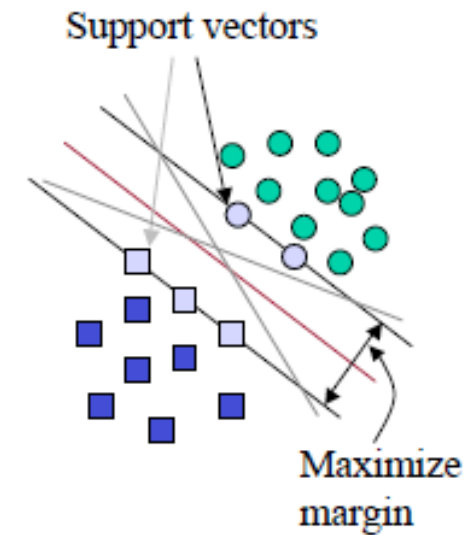
# Idea metody SVM

- Podstawowa idea stosowania SVM (support vector machines) jest podobna do sieci neuronowych jedno lub wielowarstwowych
- – optymalna hiperpłaszczyzna do liniowej separacji wzorców.
- Rozszerzenie do wzorców, które nie są liniowo separowalne poprzez transformację oryginalnych danych na nową płaszczyznę – funkcja Kernela.
- SVM algorytm do rozpoznawania wzorców.
- Support vectors czyli wektory wspierające to punkty w danych które leżą najbliżej do płaszczyzny decyzji (lub hiperpłaszczyzny). Są to punkty, które najtrudniej zaklasyfikować.
- Zależą bezpośrednio od optymalnego położenia płaszczyzny decyzji.
- Dla przestrzeni jednowymiarowej:



# SVM

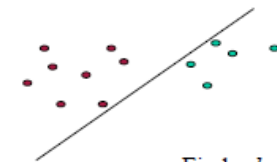
- SVM maksymalizują margines dookoła oddzielającej hiperpłaszczyzny.
- Funkcja decyzji jest wyspecyfikowana poprzez podzbiór prób treningowych tzw. wektorów wspierających.
- Problem zostaje sprowadzony do programowania kwadratowego – łatwy do rozwiązania za pomocą standardowych metod.



# Zagadnienie separacji

- Dla uproszenia zakładamy separację liniową, w dwóch wymiarach wystarczy prosta, dla większej liczby wymiarów potrzebna jest hiperpłaszczyzna.
- Wejście: zestaw danych wejściowych objaśniających:  $x_1, x_2, \dots, x_n$  oraz wynik  $y$ . Może być wiele danych wejściowych.
- Wynik: zestaw wag  $\mathbf{w}$  ( $w_i$ ), jedna dla każdego obiektu którego liniowa kombinacja przewiduje wartość  $y$ . (podobnie jak w sieciach neuronowych...)
- Podstawowa różnica: wykorzystywana jest optymalizacja maksymalizująca margines (margin) do redukcji liczby wag które są niezerowe do kilku odpowiadających istotnym obiektom decydującym o separacji prostą lub hiperpłaszczyzną. Niezerowe wagi odpowiadają wektorom wspierającym ponieważ wspierają separację hiperpłaszczyzny (stąd nazwa).

2-D Case



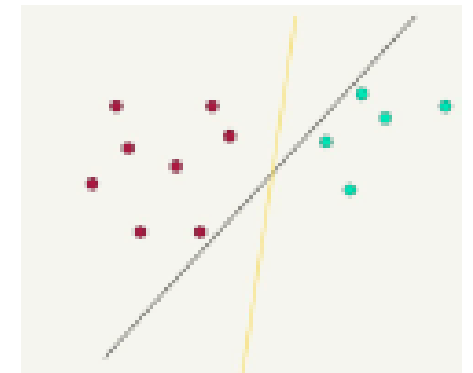
Find  $a, b, c$ , such that

$ax + by \geq c$  for red points

$ax + by \leq$  (or  $<$ )  $c$  for green points.

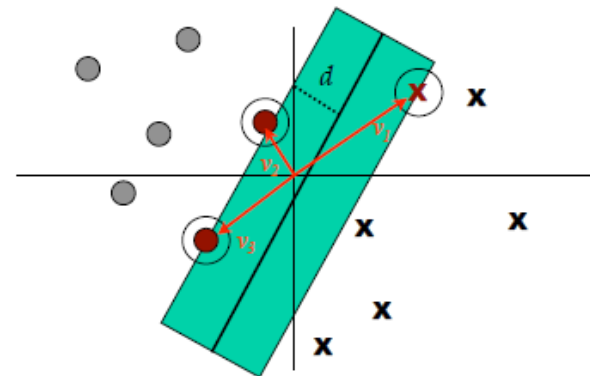
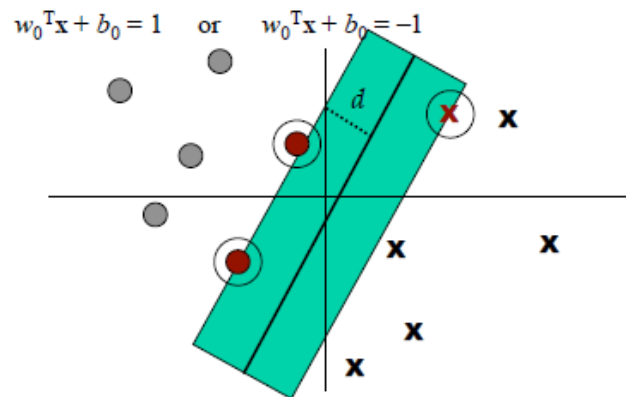
## Którą hiperpłaszczyznę wybrać?

- Wiele możliwych rozwiązań dla  $a, b, c$ .
- Są metody, które znajdują hiperpłaszczyznę ale nie optymalną (np. sieci neuronowe)
- Które punkty powinny wpływać na rozwiązanie optymalne?
- Wszystkie?
  - regresja liniowa
  - sieć neuronowa
- Czy tylko punkty problematyczne blisko granic decyzyjnych?
  - Support vector machines



# SVM dla przypadku liniowej separacji

- Wektory wspierające to elementy zbioru treningowego które w przypadku usunięcia zmieniają pozycję rozdzielającej hiperpłaszczyzny.
- Wektory wspierające są więc krytycznymi elementami zbioru treningowego.
- Problem szukania optymalnej hiperpłaszczyzny to problem optymalizacyjny który może być rozwiązany technikami optymalizacyjnymi (mnożniki Lagrange'a mogą być wykorzystane do sprowadzenia zagadnienia do postaci rozwiązywalnej analitycznie).
- Fig 1. Support Vectors: wektory stykają się z granicami (zaznaczone okręgami) .
- Fig 2. Faktyczne wektory  $v_1, v_2, v_3$ , zamiast trzech punktów styku,  $d$  oznacza 1/2 szerokości pasma.



# SVM definicje

Hiperpłaszczyzna (H) jest zdefiniowana jako:

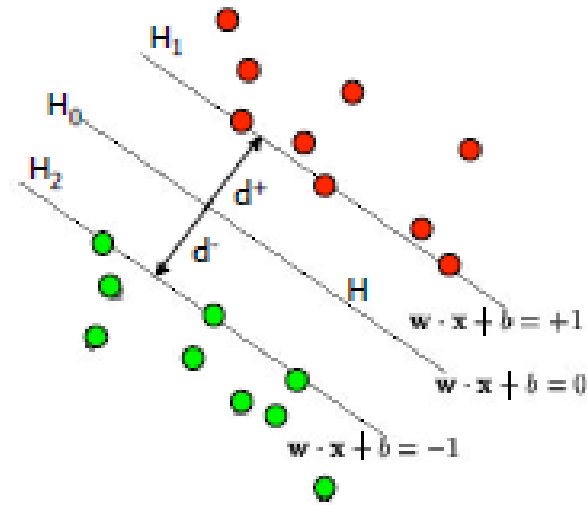
$$w \bullet x_i + b \geq +1 \text{ gdy } y_i = +1$$

$$w \bullet x_i + b \leq -1 \text{ gdy } y_i = -1$$

$H_1$  i  $H_2$  to hiperpłaszczyzny:

$$H_1: w \bullet x_i + b = +1$$

$$H_2: w \bullet x_i + b = -1$$



Punkty na płaszczyznach  $H_1$  i  $H_2$  to wyznaczniki wektorów wspierających.

Płaszczyzna  $H_0$  jest medianą pomiędzy, gdzie  $w \bullet x_i + b = 0$

$(d^+)$  = najbliższa odległość do najbliższego punktu dodatniego

$(d^-)$  = najbliższa odległość do najbliższego punktu ujemnego

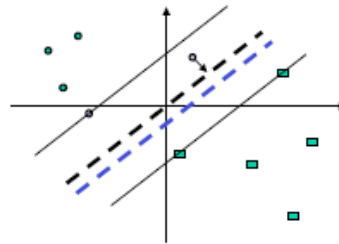
Margines hiperpłaszczyzny oddzielającej to:  $(d^+) + (d^-)$ .



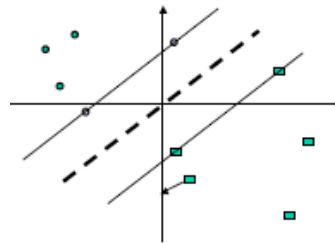
# SVM definicje

- Algorytm optymalizacyjny do wyznaczenia wag polega na tym że tylko wektory wspierające określają wagi a tym samym granice.

- Przesunięcie wektora wspierającego
- Przesuwa granice decyzyjne



- Przesunięcie innego wektora
- nie ma wpływu

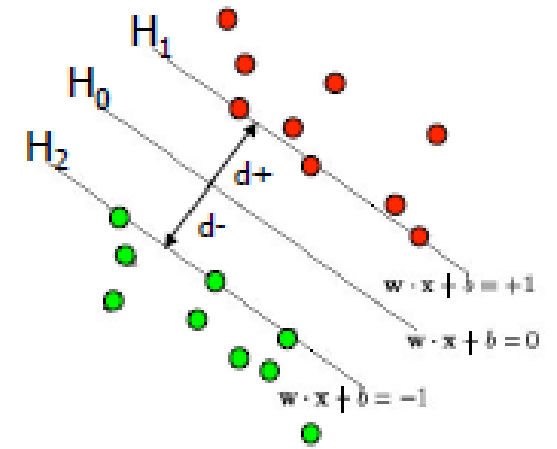


## SVM – definicje hiperprostej

- Równanie definiujące powierzchnię oddzielającą klasy jest hiperpłaszczyzna określona formą:
- $\mathbf{w}^T \mathbf{x} + b = 0$
- –  $\mathbf{w}$  wektor wag
- –  $\mathbf{x}$  wektor wejściowych zmiennych
- –  $b$  stała
- Co pozwala na zapis:
- $\mathbf{w}^T \mathbf{x} + b \geq 0$  dla  $d_i = +1$
- $\mathbf{w}^T \mathbf{x} + b < 0$  dla  $d_i = -1$
- Margines separacji ( $d$ ): oddzielenie (separacja) pomiędzy hiperpłaszczyzną i najbliższym punktem danych dla danego wektora wag  $\mathbf{w}$  i stałej  $b$ .
- Optymalna hiperpłaszczyzna (maximal margin): hiperpłaszczyzna dla której margines separacji  $d$  jest maksymalny.

# Maksymalizacja marginesu (margin)

- Požadany jest klasyfikator (liniowy separator) z tak dużym marginesem jak to tylko możliwe.
- Zdefiniujemy odległość z punktu  $(x_0, y_0)$  do prostej:
- $Ax + By + c = 0$  jako:  $|Ax_0 + By_0 + c| / \sqrt{A^2 + B^2}$ ,
- Stąd odległość pomiędzy  $H_0$  i  $H_1$ :
- $|w \cdot x + b| / ||w|| = 1 / ||w||$ ,
- Więc całkowita odległość pomiędzy  $H_1$  i  $H_2$  jest wówczas:  $2 / ||w||$
- W celu maksymalizacji marginesu, niezbędna jest minimalizacja  $||w||$ .
- (pod warunkiem że nie ma żadnego punktu danych pomiędzy  $H_1$  i  $H_2$ ):
- $x_i \cdot w + b \geq +1$  gdy  $y_i = +1$
- $x_i \cdot w + b \leq -1$  gdy  $y_i = -1$
- Może być połączone jako:  $y_i(x_i \cdot w) \geq 1$

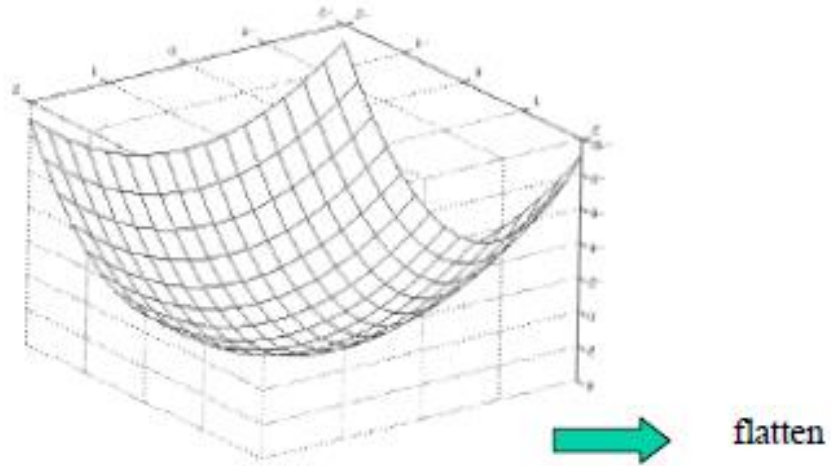


## Zagadnienie programowania kwadratowego

- Minimalizacja  $\| \mathbf{w} \|^2$ , pozwala na otrzymanie granicy dyskryminacji  $\min f(x)$  s.t.  $g(x)=0$ , zapisanej jako:
- $\min f: \frac{1}{2} \| \mathbf{w} \|^2$  (funkcja kwadratowa)
- Stąd rozwiązaniem jest:  $\mathbf{y}_i(\mathbf{w} \bullet \mathbf{x}_i) - b = 1$  lub  $[\mathbf{y}_i(\mathbf{w} \bullet \mathbf{x}_i) - b] - 1 = 0$
- Czyli problem optymalizacji z ograniczeniami
- Może być rozwiązany z wykorzystaniem metody mnożnika Lagrange'a
- Ponieważ jest to funkcją kwadratowa to płaszczyzna jest w kształcie paraboloidy z jednym minimum globalnym (czyli unikamy problemu występującego w sieciach neuronowych!)

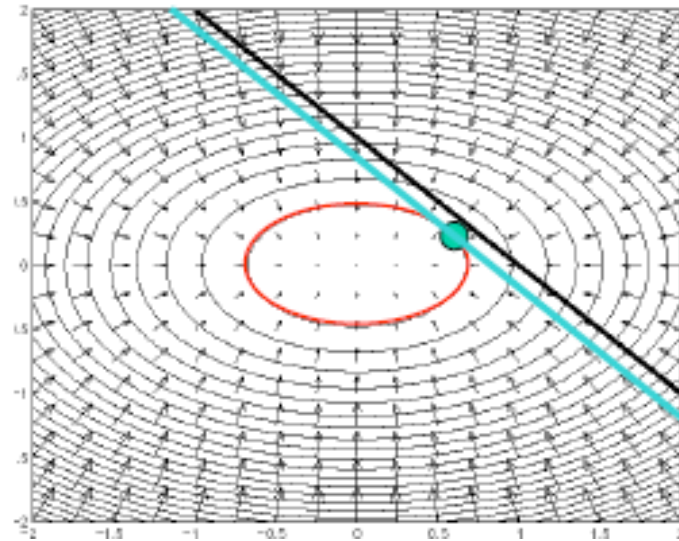
# Przykład

- Przykład: paraboloida  $2+x^2+2y^2$  s.t.  $x+y=1$
- Intuicja: znajdź przecięcie dwóch funkcji  $f, g$  w punkcie styczności (przecięcie = obydwa warunki spełnione; styczność = pochodna wynosi 0);
- to będzie min (lub max) dla  $f$  czyli warunek  $g$  jest spełniony.



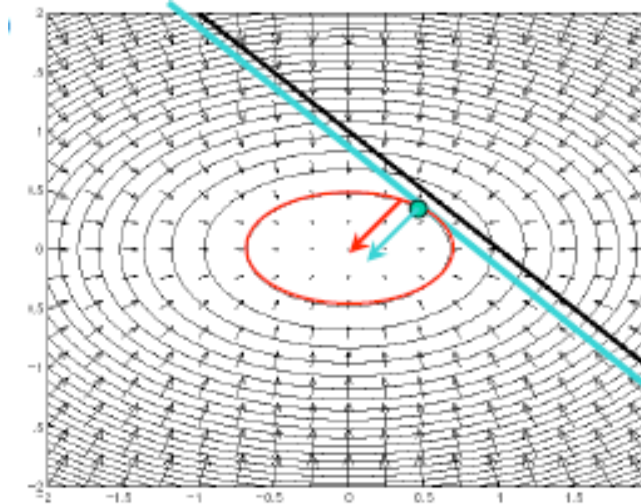
# Przykład

- Spłaszczona paraboloida  $f: 2x^2+2y^2=0$  z dodatkowym ograniczeniem  $g: x+y=1$
- *Minimalizacja* kiedy prosta ograniczająca  $g$  (zielona) jest styczna do wewnętrznej elipsy  $f$  (czerwona) – kierunek wewnętrznych strzałek.



# Przykład

- Spłaszczona paraboloida  $f: 2x^2+2y^2=0$  z dodatkowym ograniczeniem  $g: x+y=1$
- W stycznym rozwiązaniu  $p$ , styczne wektory  $f, g$  są równoległe (nie ma możliwości zwiększenia  $f$  przy zachowaniu obszaru  $g$ )
- *Minimalizacja* kiedy prosta ograniczająca  $g$  jest styczna do wewnętrznej elipsy  $f$



# Przykład

- Dwa ograniczenia
- 1. równoległe normalne ograniczenie (= styczne ograniczenie na  $f$ ,  $g$  rozwiązaniem jest max lub min)
- 2.  $g(x)=0$  (rozwiązaniem jest również prosta ograniczająca)
- Można przekształcić poprzez połączenie  $f$ ,  $g$  jako nowej funkcji Lagrange'a poprzez wprowadzenie nowej zmiennej oznaczonej jako  $a$  (częściej oznaczanej jako  $\alpha$ ) lub poprzez opisanie tych warunków.
- Punkt rozwiązania  $p$  gdzie:
$$\nabla f(p) = \nabla \lambda g(p)$$
$$g(x) = 0$$
- Lub połączenia jako funkcja Lagrange'a  $L$  i porównanie pochodnej do 0 :
$$L(x, a) = f(x) - ag(x)$$
$$\nabla L(x, a) = 0$$



# Przykład

- W punkcie rozwiązania  $p$
- • prosta ograniczająca  $g$  oraz kontur  $f$  muszą być styczne
- • jeśli są styczne ich wektory nachylone są równoległe
- • nachylenie  $g$  musi być 0 – czyli, najbardziej strome nachylenie i więc równoległy do  $f$
- • nachylenie  $f$  musi być w tym samym kierunku jak  $g$
- Jak funkcja Lagrange'a rozwiązuje ograniczona optymalizację:

- Częściowe pochodne wrt  $x$  znajdują równoległe ograniczenia normalne

- Częściowe pochodne wrt  $\lambda$  znajdują  $g(x,y)=0$

- Ogólnie:  $L(x, a) = f(x) - a g(x)$  where

$$\nabla(x, a) = 0$$

$$L(x, a) = f(x) + \sum_i a_i g_i(x)$$

- Uogólniając:  $L(x, a) = f(x) + \sum_i a_i g_i(x)$  jest funkcją  $n+m$  zmiennych  $n$  dla  $x$  natomiast  $m$  dla  $a$ .
- Pochodna daje  $n+m$  równań, każde przyrównane do 0 ( $n$  daje warunki gradientowe, natomiast  $m$  ograniczenia).

$$f(x): \frac{1}{2} \| \mathbf{w} \|^2 ; g(x): y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 = 0$$

- więc Lagrange wynosi:

$$\min L = \frac{1}{2} \| \mathbf{w} \|^2 - \sum a_i [y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1] \text{ wrt } \mathbf{w}, b$$

- lub w innej formie:

$$\min L = \frac{1}{2} \| \mathbf{w} \|^2 - \sum a_i y_i(\mathbf{w} \cdot \mathbf{x}_i + b) + \sum a_i \text{ wrt } \mathbf{w}, b$$

# $L_p$ – funkcja Lagrange’a w SVM

Formuła Lagrange’a w problemie SVM:

$$L_p = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^l a_i y_i (\mathbf{x}_i \mathbf{w} + b) + \sum_{i=1}^l a_i$$

$\forall_i, a_i \geq 0$  gdzie  $l$  jest liczbą punktów treningowych

Z warunku pochodnej w minimum =0 otrzymujemy:

$$\frac{\partial L_p}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^l a_i y_i \mathbf{x}_i = 0$$
$$\frac{\partial L_p}{\partial b} = \sum_{i=1}^l a_i y_i = 0 \quad \text{więc}$$
$$\mathbf{w} = \sum_{i=1}^l a_i y_i \mathbf{x}_i, \sum_{i=1}^l a_i y_i = 0$$

# $L_p$ – funkcja Lagrange’a w SVM

- Jest to podstawowa forma problemu optymalizacyjnego w SVM.
- Problem optymalizacji zostanie rozwiązany poprzez rozwiązanie podwójnego problemu oryginalnego.
- Podwójny problem Lagrange’a jest sformułowany jako:
- Zamiast minimalizacji po  $\mathbf{w}$ ,  $b$ , przy warunkach ograniczających zawierających  $a$ , możemy maksymalizować po  $a$  (podwójna zmienna) przy relacjach otrzymanych poprzednio dla  $\mathbf{w}$  i  $b$ .
- Rozwiązanie musi spełniać dwa warunki:
$$\mathbf{w} = \sum_{i=1}^l a_i y_i \mathbf{x}_i, \quad \sum_{i=1}^l a_i y_i = 0$$
- Poprzez podstawienie  $\mathbf{w}$  i  $b$  z powrotem w oryginalnym równaniu pozbywamy się zależności od  $\mathbf{w}$  i  $b$ .
- Czyli wagi  $\mathbf{w}$  muszą być liniową kombinacją wejściowych i wyjściowych danych treningowych  $\mathbf{x}_i$  i  $y_i$  oraz wartości  $a$ . Teraz należy rozwiązać po  $a$  poprzez pochodną i porównanie do 0. Większość z  $a$  będzie miała wartość 0, natomiast niezerowe będą odpowiadały wektorom wspierającym.

Pierwszy problem:

$$\min L_p = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^l a_i y_i (\mathbf{x}_i \mathbf{w} + b) + \sum_{i=1}^l a_i$$

Gdzie:

$$\forall_i a_i \geq 0$$

$$w = \sum_{i=1}^l a_i y_i x_i, \quad \sum_{i=1}^l a_i y_i = 0$$

Drugi problem:

$$\max L_D(a_i) = \sum_{i=1}^l a_i - \frac{1}{2} \sum_{i=1}^l a_i a_j y_i y_j (\mathbf{x}_i \mathbf{x}_j)$$

Gdzie:

$$\sum_{i=1}^l a_i y_i = 0 \text{ oraz } a_i \geq 0$$

Pochodna po  $a$  porównana do 0 daje rozwiązanie dla  $a_i$ :

$$\sum_{i=1}^l a_i y_i = 0 \quad 0 \leq a_i \leq C$$

# Podwójny problem lagrange'a

- Rozwiązanie będzie takie samo jak rozwiązanie wyjściowego problemu.
- Rozwiązanie podwójnego problemu jednak jest przydatne na późniejszym etapie kiedy chcemy rozwiązać nieliniową separację w problemie klasyfikacyjnym.
- Znając  $a_i$  można wyznaczyć wagi  $w$  dla maksymalnego marginesu oddzielającej hiperpłaszczyzny:
- $\mathbf{w} = \sum_{i=1}^l a_i y_i \mathbf{x}_i$
- Po wyznaczeniu wag  $\mathbf{w}$ , mając nieznany punkt  $u$  z wartościami  $x_i$  można zaklasyfikować go poprzez odczytanie znaku funkcji:
- $f(x) = \mathbf{w}\mathbf{u} + b = \left(\sum_{i=1}^l a_i y_i \mathbf{x}_i \mathbf{u}\right) + b$
- Większość wag  $w_i$  będzie wynosiła 0 tylko wektory wspierające będą miały niezerowe wagi na zmiennych a – co oznacza redukcję wymiarowości rozwiązania.

# Iloczyn skalarny, podobieństwa, i SVM

- Dlaczego iloczyn skalarny kernela powinien być stosowany do rozpoznawania wzorców z wykorzystaniem SVM ,czy w ogóle powinien być stosowany?
- Iloczyn skalarny dostarcza informacji o podobieństwie,
- Iloczyn skalarny w wymiarze 2D pomiędzy dwoma wektorami o jednostkowej długości zwraca cosinus kata pomiędzy nimi czyli oznacza jak daleko od siebie są te wektory.
- $\mathbf{x} = [1, 0]^T$  ,  $\mathbf{y} = [0, 1]^T$
- Jeśli są równoległe to iloczyn skalarny wynosi 1 (całkowite podobieństwo)
- $\mathbf{x}^T \mathbf{y} = \mathbf{x} \bullet \mathbf{y} = 1$
- Jeśli są prostopadłe (całkowite niepodobieństwo) ich iloczyn skalarny wynosi 0 (czyli nie powinny wchodzić do klasyfikatora)
- $\mathbf{x}^T \bullet \mathbf{y} = \mathbf{x} \bullet \mathbf{y} = 0$

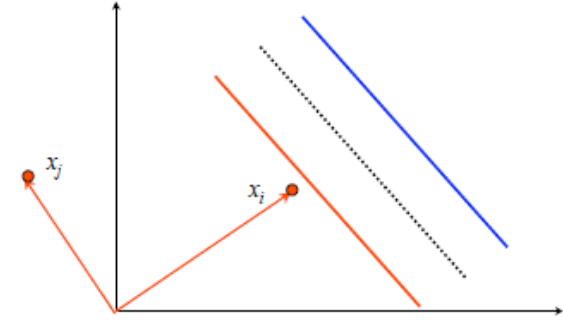
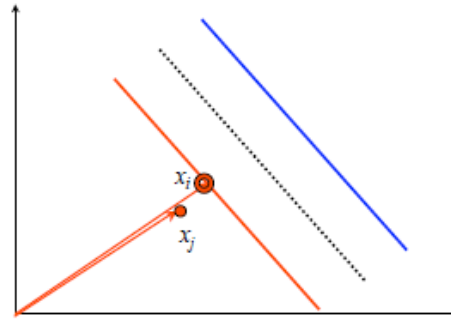
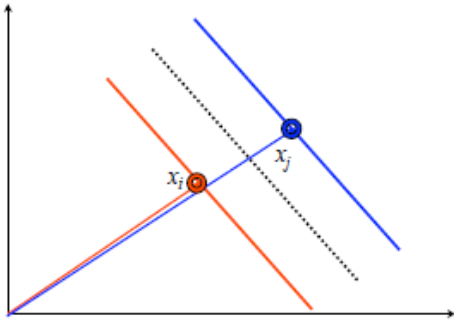
# Iloczyny skalarne

- Maksymalizujemy: 
$$L_D(a_i) = \sum_{i=1}^l a_i - \frac{1}{2} \sum_{i=1}^l a_i a_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)$$
  
s.t.  $\sum_{i=1}^l a_i y_i = 0$  &  $a_i \geq 0$
- Funkcja będzie posiadała maksimum jeśli ustawimy niezerowe wartości do zmiennych  $a$ , które odpowiadają wektorom wspierającym czyli mają znaczenie w ustawieniu szerokości marginesów. Rozpatrzmy kilka przypadków:
- Przypadek 1. Jeśli  $x_i$ ,  $x_j$  są zupełnie niepodobne, ich iloczyn skalarny wynosi 0, czyli nic nie wnoszą do  $L$ .
- Przypadek 2. Jeśli  $x_i$ ,  $x_j$  są zupełnie podobne, ich iloczyn skalarny wynosi 1. Rozróżniamy dwa warianty:
- Wariant 1: oba  $x_i$  i  $x_j$  przewidują ten sam wynik  $y_i$  (+1 lub -1). Czyli  $y_i \cdot y_j$  wynosi zawsze 1, i wartość  $a_i a_j y_i y_j x_i x_j$  będzie dodatnia. Ale to obniży wartość  $L$  (ponieważ jest wyłączone z pierwszej części sumy). Czyli algorytm obniża podobne wektory które dają tą samą predykcję.
- Wariant 2:  $x_i$  i  $x_j$  przewidują przeciwny wynik  $y_i$  (jeden jest +1, drugi jest -1), ale są bardzo podobne: wówczas  $a_i a_j y_i y_j x_i x_j$  jest ujemny i wyłączone czyli dodaje się do sumy maksymalizując ją. Czyli przykład którego szukamy: pokazujący różnice między klasami.



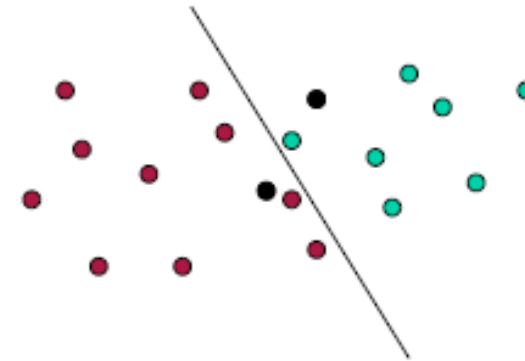
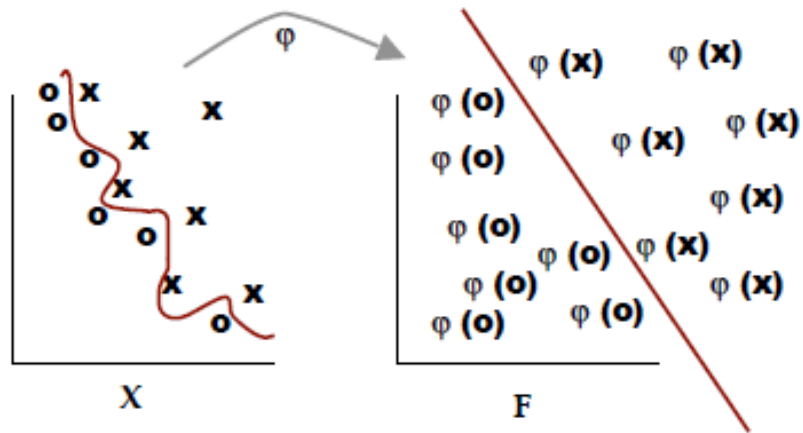
# Iloczyny skalarne – graficznie

- 2 bardzo podobne  $x_i, x_j$  wektory do predykcji różnych klas maksymalizują szerokość marginesu
- 2 wektory które są podobne ale przewidują te same klasy są redundantne
- 2 różniące się wektory (prostopadłe) w ogóle nie są brane pod uwagę



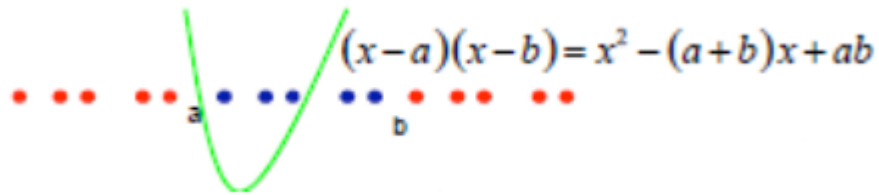
# Nieliniowa separacja!

- Szukana jest prosta (lub krzywa), która nadaje karę punktom które znajdują się po „niewłaściwej” stronie
- Do podziału dokonujemy transformacji:

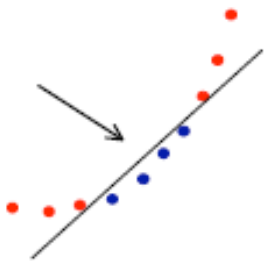


# Nieliniowe wektory wspierające SVMs

- Idea metody polega na otrzymaniu liniowej separacji poprzez mapowanie danych do przestrzeni o wyższym wymiarze.
- – Taki zestaw danych nie może być rozdzielony poprzez funkcję liniową ale może być poprzez kwadratową:

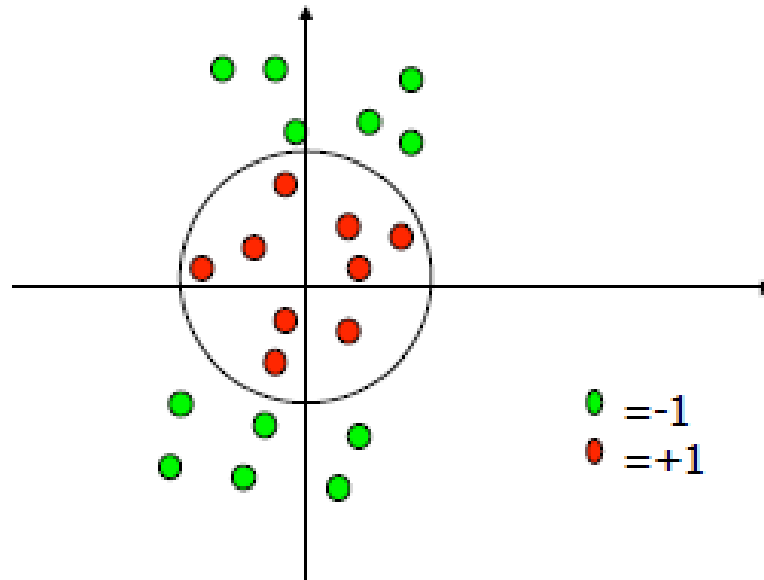


- – więc jeśli zmapujemy  $x \rightarrow \{x^2, x\}$  otrzymamy liniową separację:



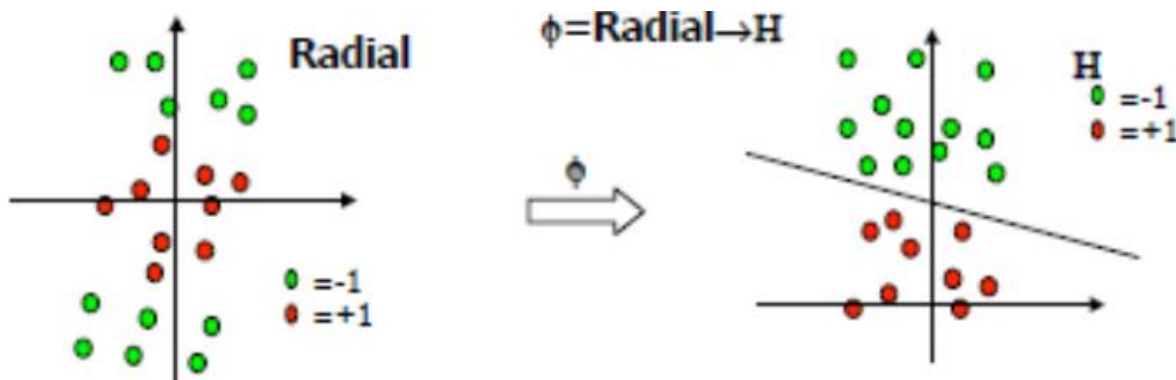
# Problemy z liniowymi wektorami wspierającymi SVM

- Co w sytuacji kiedy funkcja decyzyjną nie jest liniowa? Jaka transformacja wówczas oddzieli poniższe?



# Współrzędne biegunowe. Nieliniowe SVM.

- Trik kernela polega na funkcji  $\phi$  która mapuje dane na inną przestrzeń:



- Funkcja podlegająca optymalizacji:  $L_d = \sum a_i - \frac{1}{2} \sum a_i a_j y_i y_j (x_i \cdot x_j)$  gdzie  $(x_i \cdot x_j)$  jest iloczynem skalarnym dwóch wektorów. Jeśli zamiast wyznaczać iloczynu skalarnego dokonamy transformacji to konieczne jest wyznaczenie:  $(\phi(x_i) \cdot \phi(x_j))$ . Jest to czasochłonne i skomplikowane więc lepiej zastosować własność:
- Jeśli istnieje funkcja kernela  $K$  taka że:  $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$  to nie ma potrzeby wyznaczania  $\phi$  w ogóle ponieważ funkcja kernela określa iloczyn skalarny w transformowanej przestrzeni lub definiuje podobieństwa w transformowanej przestrzeni.

# Nieliniowe wektory wspierające SVMs

- Funckja otrzymana z optymalizacji:  $L_d = \sum a_i - 1/2 \sum a_i a_j y_i y_j K(x_i \cdot x_j)$
- Przykład kernela: kernel wielomianowy  $K(x_i, x_j) = (x_i \bullet x_j + 1)^p$ , gdzie  $p$  jest parametrem dostrajającym.

# Przykłady nieliniowych wektorów wspierających SVMs

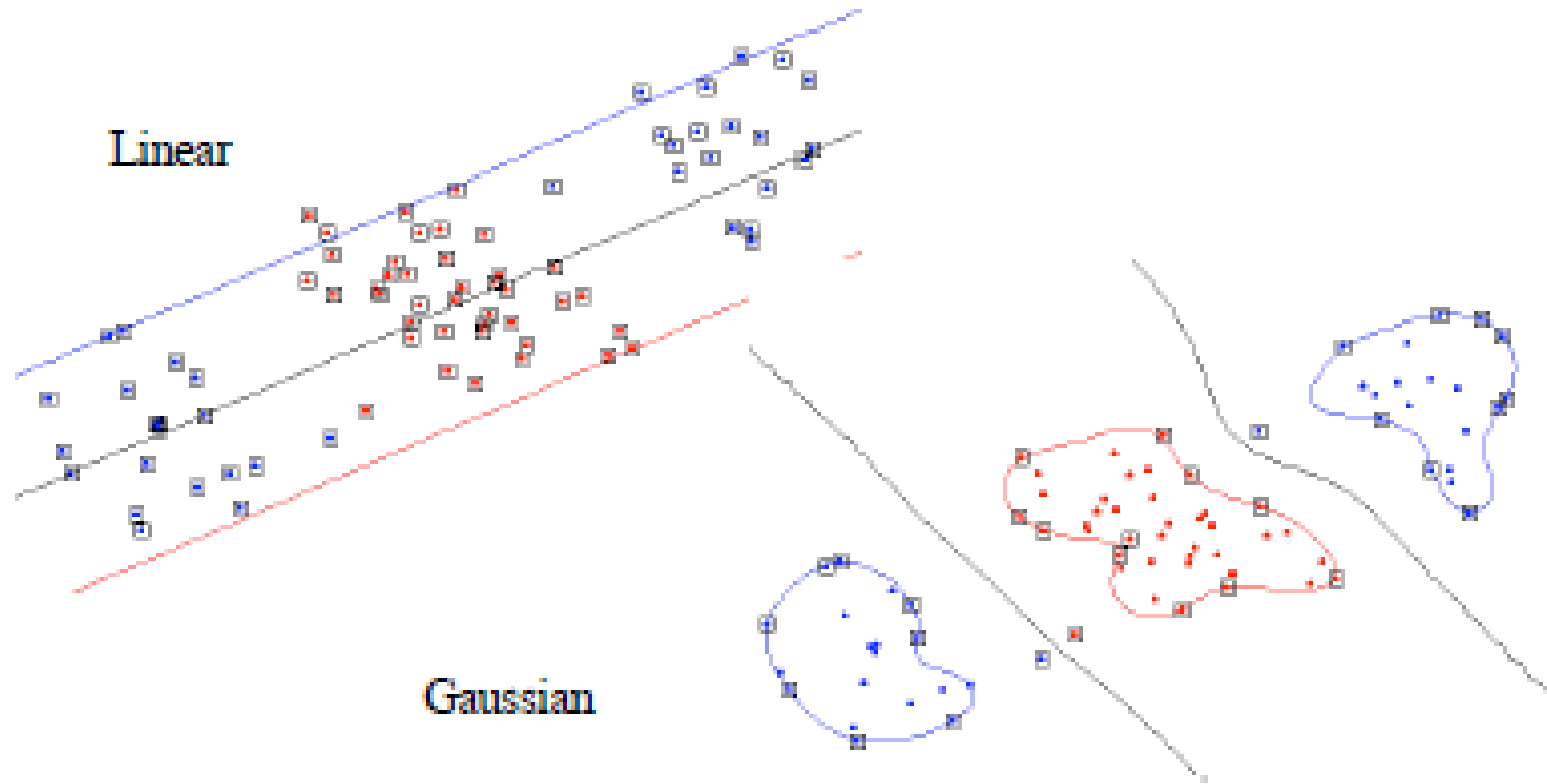
- Wielomianowy (włączając  $\mathbf{x} \cdot \mathbf{x}$  jako przypadek szczególny)  $K(\mathbf{x}, \mathbf{y}) = (\mathbf{x} \cdot \mathbf{y} + 1)^p$
- Funkcja radialna (gaussa)  $K(\mathbf{x}, \mathbf{y}) = \exp \left\{ -\frac{\|\mathbf{x} - \mathbf{y}\|^2}{2\sigma^2} \right\}$
- Sigmoidalna (funkcja aktywacji w sieciach neuronowych)  $K(\mathbf{x}, \mathbf{y}) = \tanh(\kappa \mathbf{x} \cdot \mathbf{y} - \delta)$

# Iloczynny skalarne Kernela

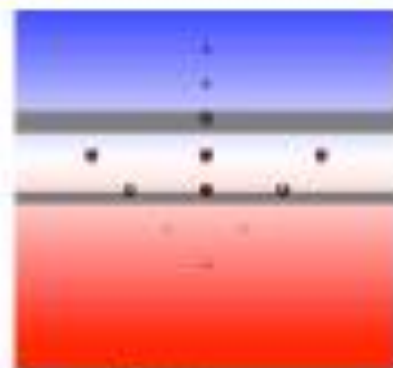
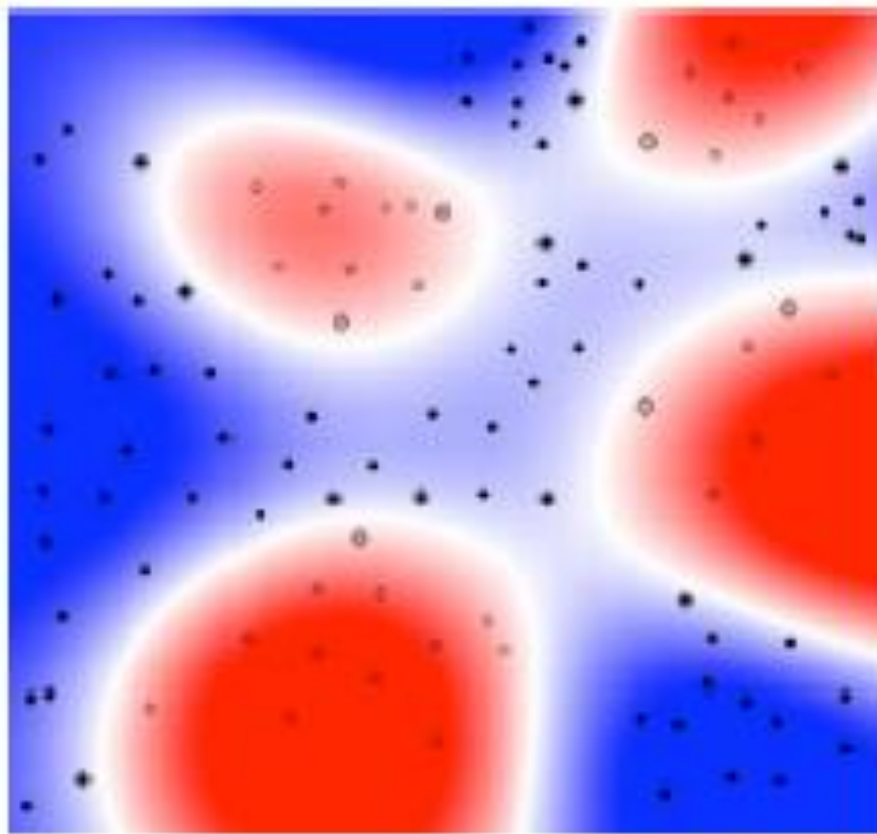
|                                |                                                         |                                                                |
|--------------------------------|---------------------------------------------------------|----------------------------------------------------------------|
| Type of Support Vector Machine | Inner Product Kernel<br>$K(x, x_i), i = 1, 2, \dots, N$ | Usual inner product                                            |
| Polynomial learning machine    | $(x^T x_i + 1)^p$                                       | Power $p$ is specified <i>a priori</i> by the user             |
| Radial-basis function (RBF)    | $\exp(1/(2\sigma^2) \ x - x_i\ ^2)$                     | The width $\sigma^2$ is specified <i>a priori</i>              |
| Two layer neural net           | $\tanh(\beta_0 x^T x_i + \beta_1)$                      | Actually works only for some values of $\beta_0$ and $\beta_1$ |



# Przykłady nieliniowych wektorów wspierających – gaussowski Kernel



# Nieliniowy radialny kernel



linear



second order polynomial



third order polynomial



radial basis, 2.0



radial basis, 0.5



radial basis, 0.08